

NP-Completeness, NP-hardness for Optimization

Techniques for reductions, Proof writing guide, NP-hard optimization problems

Last Lecture

- **P**: decision problems that can be **decided** efficiently
- **NP**: decision problems that can be **verified** efficiently
- **Reduction** is an effective tool to show one problem is "weakly harder" than another.
- **NP-Completeness** describes the hardest problems in **NP**.
- Cook-Levin Theorem. **SAT** is NP-complete.
- **3SAT**, **VertexCover**, **IndependentSet**, **SubsetSum**, **HamiltonianPath** are NP-complete.

This Lecture

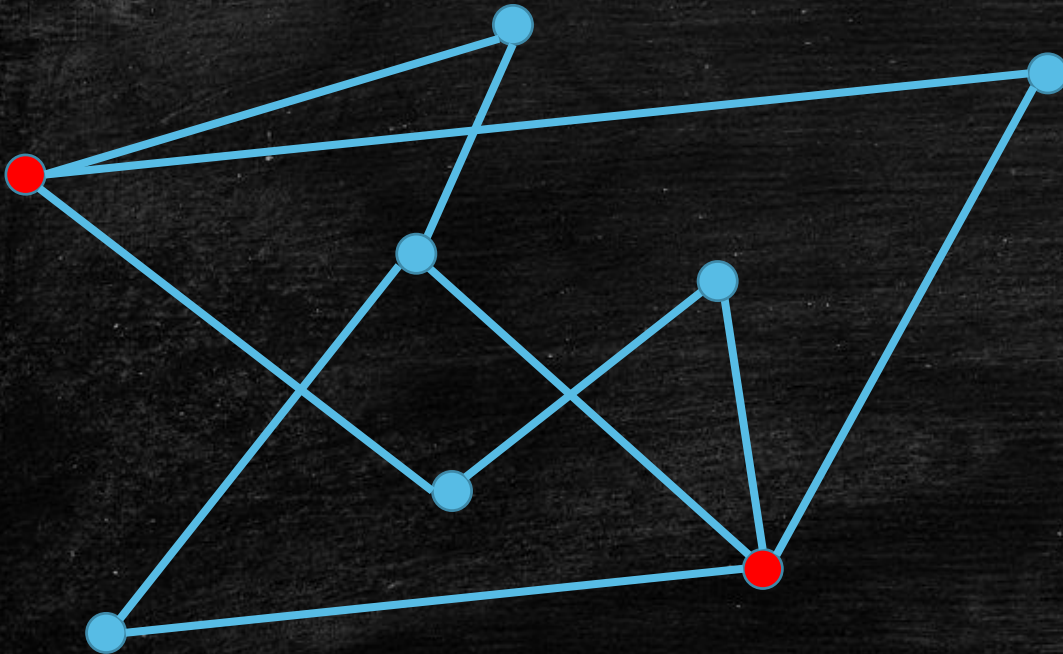
- Show more important NP-complete problems.
- Learn some elementary techniques for reduction.
- Learn how to write a formal proof for NP-completeness.
- NP-hard optimization problems.

Note 1: Choose the Right Problem to Reduce from.

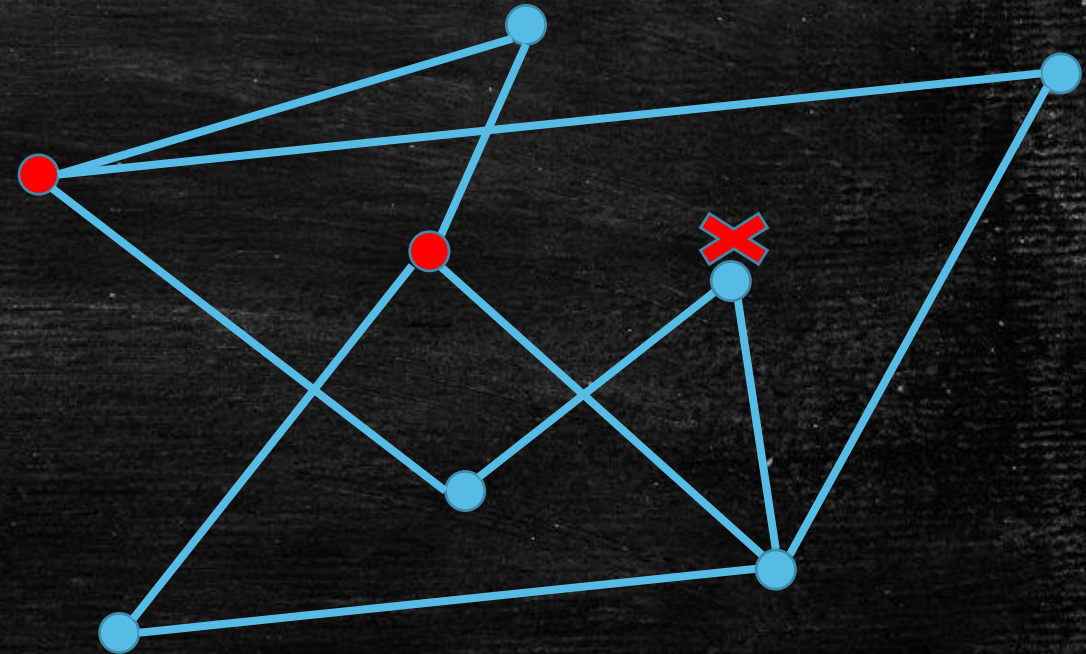
- Want to show an **NP** problem f is NP-complete.
- Need to show $g \leq_k f$ for some NP-complete problem g .
- Conceptually and in principle, $g \leq_k f$ should hold for **any** NP problem g .
 - Choosing **any** NP-complete problem should work, e.g., SAT.
- However, choosing a suitable problem makes your life much easier!
- If possible, choose g that “looks similar to” f .

Dominating Set

- Given an undirected graph $G = (V, E)$, a **dominating set** is a subset of vertices S such that, for any $v \in V \setminus S$, there is a vertex $u \in S$ that is adjacent to v .



a dominating set

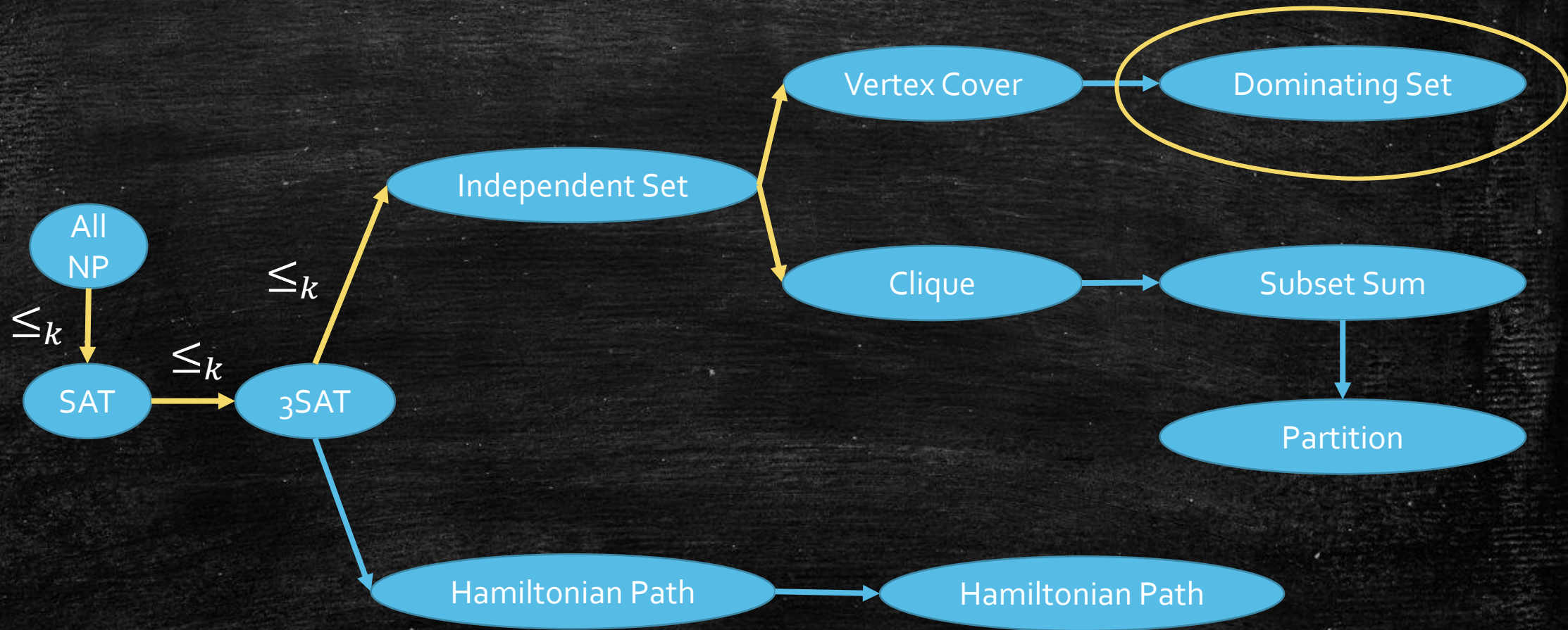


not a dominating set

Dominating Set Problem

- **[DominatingSet]** Given an undirected graph $G = (V, E)$ and an integer $k \in \mathbb{Z}^+$, decide if G contains a dominating set with size k .
- Problem: Show that DominatingSet is NP-complete.
- Question: Which problem should we reduce from?

Our Reduction Graph



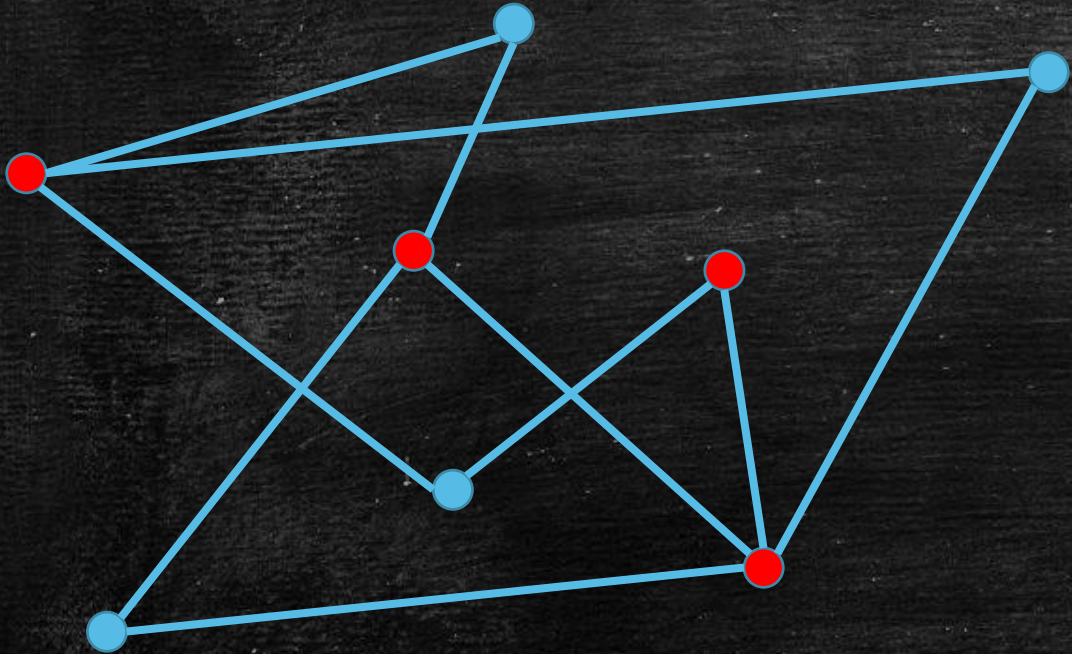
Reduction from VertexCover

- A dominating set is similar to a vertex cover:
 - Vertex cover: S covers edges
 - Dominating set: S covers vertices
- An idea for reduction:
 - Introduce an intermediate vertex for each edge
 - cover the edge $\Rightarrow$ cover the intermediate vertex

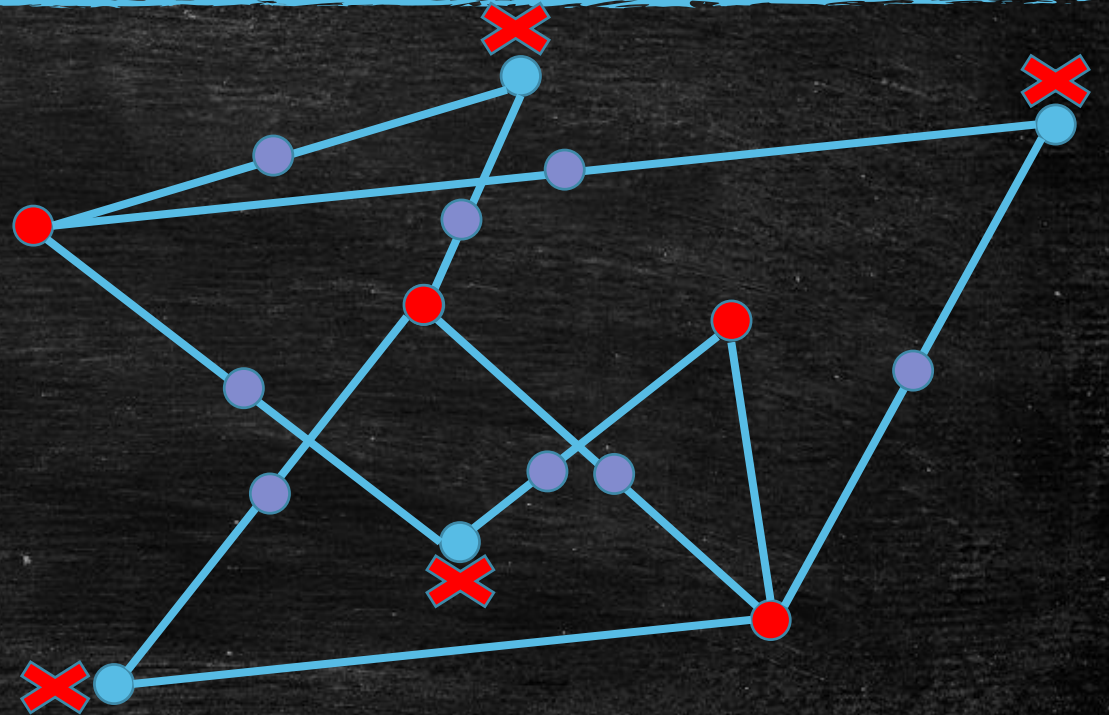


Does it work?

Does it work? **NO!**



a vertex cover

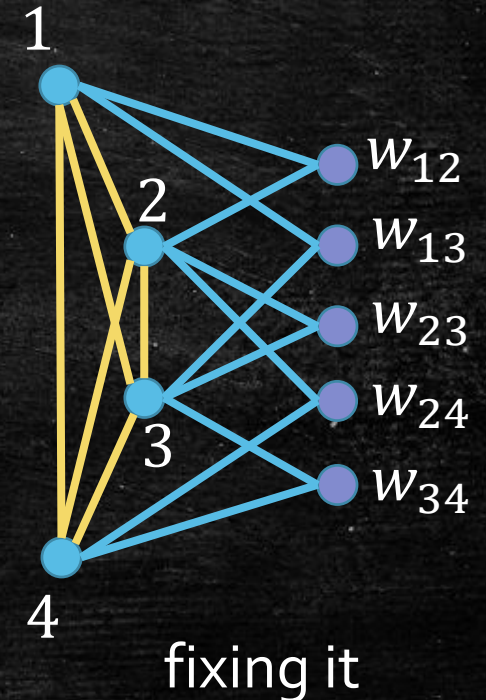
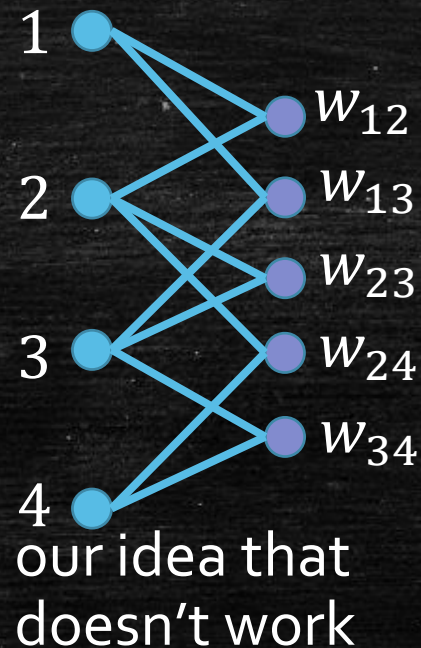
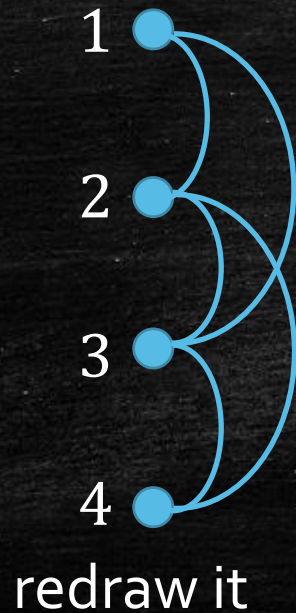
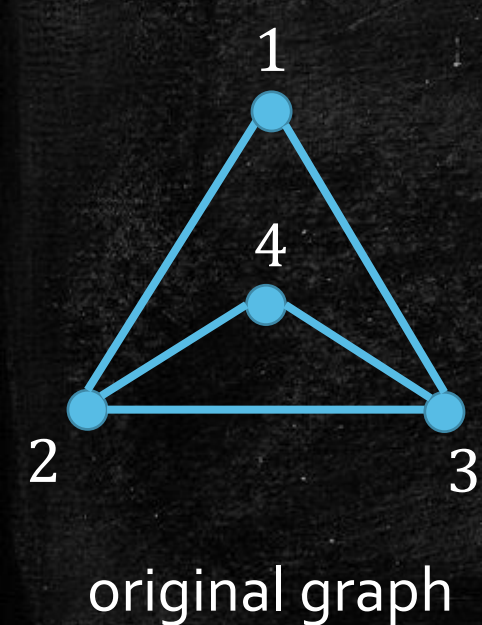


not a dominating set

- New vertices are covered, but original vertices may not be covered!
- Can we fix it?

Note 2: Fix your reduction if it doesn't work.

- All we have to do: make the original vertices a **clique**!
- Now, selecting a single vertex in the original vertex set covers all the original vertices.



How to write a NP-Completeness Proof

Four Parts for proving f is NP-complete:

1. Prove that f is in **NP**
2. Present the reduction $g \leq_k f$ for an NP-complete problem g
3. Show that yes instances of g are mapped to yes instances of f
4. Show that no instances of g are mapped to no instances of f
 - Most of the time, it is easier to prove its contrapositive: if an instance x of g is mapped to a yes instance of f , then x is a yes instance of g .

DominatingSet is NP-complete

– a formal proof

Proof. First of all, DominatingSet is in **NP**, as a dominating set S can be served as a certificate, and it can be verified in polynomial time whether S is a dominating set and whether $|S| = k$.

To show that DominatingSet is NP-complete, we present a reduction from VertexCover. Given a VertexCover instance $(G = (V, E), k)$, we construct a DominatingSet instance $(G' = (V', E'), k')$ as follows.

The vertex set is $V' = \bar{V} \cup \bar{E}$, which is defined as follows. For each vertex $v \in V$ in the VertexCover instance, construct a vertex $\bar{v} \in \bar{V} \subseteq V'$; for each edge $e \in E$ in the VertexCover instance, construct a vertex $w_e \in \bar{E} \subseteq V'$.

The edge set E' is defined as follows. For each edge $e = (u, v)$ in the VertexCover instance, build two edges $(\bar{u}, w_e), (\bar{v}, w_e) \in E'$. For any two vertices $\bar{u}, \bar{v}$ in $\bar{V}$, build an edge $(\bar{u}, \bar{v})$.

Define $k' = k$.

DominatingSet is NP-complete – a formal proof (continued)

Proof (Continued).

Suppose $(G = (V, E), k)$ is a yes VertexCover instance. There exists a vertex cover $S \subseteq V$ with $|S| = k$. We will prove $\bar{S}$ corresponding S is a dominating set in G' .

For each vertex in $\bar{V}$, it is covered by any vertex in $\bar{S}$ as $\bar{V}$ forms a clique.

For each vertex w_e in $\bar{E}$, let $e = (u, v) \in E$ be the corresponding edge in the VertexCover instance. We have either $u \in S$ or $v \in S$ (or both), as S is a vertex cover. This implies either $\bar{u} \in \bar{S}$ or $\bar{v} \in \bar{S}$ (or both), which further implies w_e is covered as $(\bar{u}, w_e), (\bar{v}, w_e) \in \bar{E}$ by our construction.

Since $|\bar{S}| = |S| = k = k'$, the DominatingSet instance we constructed is a yes instance.

DominatingSet is NP-complete – a formal proof (continued)

Suppose $(G' = (V', E'), k')$ is a yes DominatingSet instance. There exists a dominating set $S' \subseteq V' = \bar{V} \cup \bar{E}$ with $|S'| = k' = k$. We aim to show that $(G = (V, E), k)$ is a yes VertexCover instance.

First of all, we can assume $S' \subseteq \bar{V}$ without loss of generality. If we have $w_e \in S'$ for some $w_e \in \bar{E}$, we can replace w_e with either $\bar{u}$ or $\bar{v}$ for the edge $e = (u, v)$ in the VertexCover instance. (In the case $\bar{u}$ and $\bar{v}$ have already been included in S' , we can replace w_e with any unpicked vertex in $\bar{V}$.) It is easy to see that S' is still a dominating set after the change, as the set of vertices covered by either $\bar{u}$ or $\bar{v}$ is a superset of the set of vertices covered by w_e (which is just $\{\bar{u}, \bar{v}\}$).

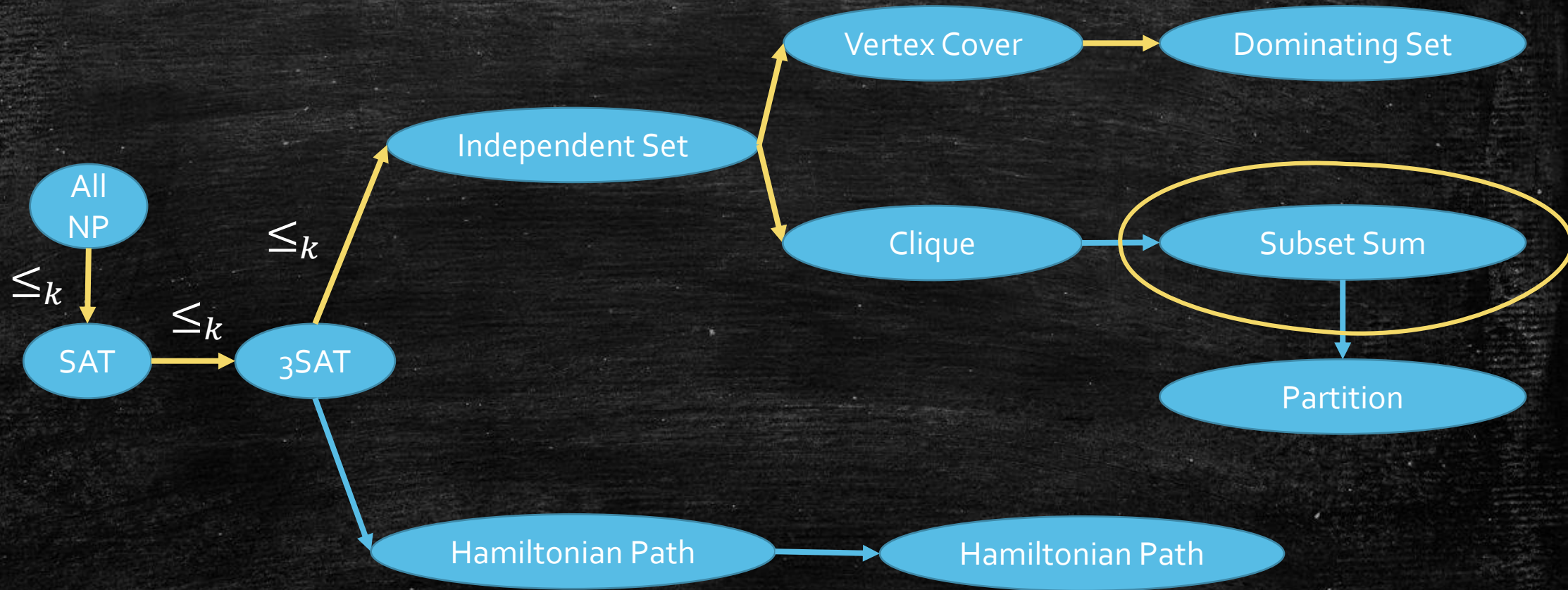
Next, since $S' \subseteq \bar{V}$, S' corresponds to a vertex set $S \subseteq V$ in the VertexCover instance with $|S| = |S'| = k' = k$. It remains to show S is a vertex cover.

For any edge $e = (u, v)$, we have either $\bar{u} \in S'$ or $\bar{v} \in S'$ (or both) since S' is a dominating set and $\bar{u}, \bar{v}$ are the only two vertices that can cover w_e . This implies $u \in S$ or $v \in S$ (or both), so S is a vertex cover.

Some Additional Notes

- Note 3: To prove a no instance is mapped to a no instance, we often prove the contrapositive.
- Note 4: When proving the above-mentioned contrapositive for $g \leq_k f$, a common technique is to show that we can assume the yes instance of f is “well-behaved” that corresponds to the yes instance of g .
 - E.g., we prove that we can assume $S' \subseteq \bar{V}$ just now.
- Note 5: Do not mess up with the direction: a common mistake is to construct a instance of g from f , which only shows $f \leq_k g$ (which is not helpful).

Our Reduction Graph



Note 6: Find an intermediate problem

- $g \leq_k f$ where g and f look quite different.
- Find an intermediate problem h that has similarities to both g and f .
- Show that $g \leq_k h$ and $h \leq_k f$.

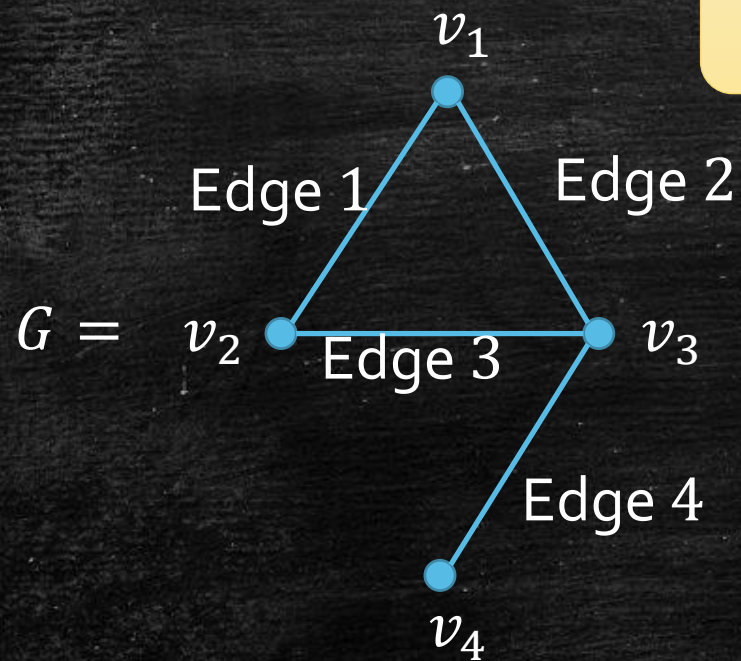
VertexCover $\leq_k$ SubsetSum

- We first consider the following “vector version” of SubsetSum.
- **[VectorSubsetSum]** Given a collection of integer vectors $S = \{\mathbf{a}_1, \dots, \mathbf{a}_n : \mathbf{a}_i \in \mathbb{Z}^m\}$ and a vector $\mathbf{k} \in \mathbb{Z}^m$, decide if there exists $T \subseteq S$ with $\sum_{\mathbf{a}_i \in T} \mathbf{a}_i = \mathbf{k}$.
- We will show that
 1. VertexCover $\leq_k$ VectorSubsetSum
 2. VectorSubsetSum $\leq_k$ SubsetSum

VertexCover $\leq_k$ VectorSubsetSum

- Task
 - Given a Vertex Cover instance $(G = (V, E), k)$.
 - Construct a Vector Subset Sum instance $(S, \mathbf{k})$.
- For each $v_i \in V$, construct a $(|E| + 1)$ -dimensional vector $\mathbf{a}_i \in S$ such that $\mathbf{a}_i[0] = 1$ and for each $j = 1, \dots, |E|$:
$$\mathbf{a}_i[j] = \begin{cases} 1 & \text{if } v_i \text{ is an endpoint of edge } j \\ 0 & \text{otherwise} \end{cases}$$
- Let $\mathbf{k} = (k, 1, 1, \dots, 1)$.

Example



We should choose
three vertices

$$k = 3$$

a Vertex Cover instance

$$\mathbf{a}_1 = (1, 1, 1, 0, 0)$$

$$\mathbf{a}_2 = (1, 1, 0, 1, 0)$$

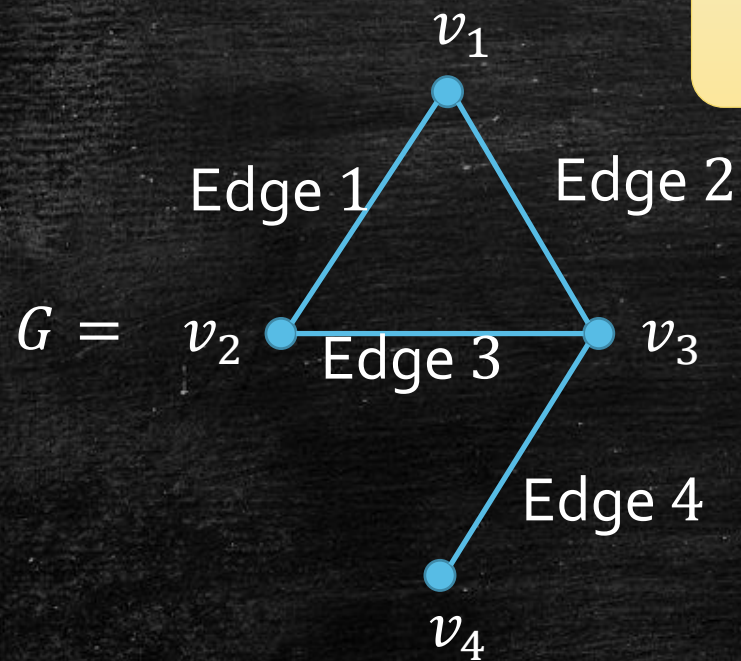
$$\mathbf{a}_3 = (1, 0, 1, 1, 1)$$

$$\mathbf{a}_4 = (1, 0, 0, 0, 1)$$

$$\mathbf{k} = (3, 1, 1, 1, 1)$$

a Vector Subset Sum instance

Example



We should choose
one vertex!

$$k = 3$$

a Vertex Cover instance

$$\mathbf{a}_1 = (1, 1, 1, 0, 0)$$

$$\mathbf{a}_2 = (1, 1, 0, 1, 0)$$

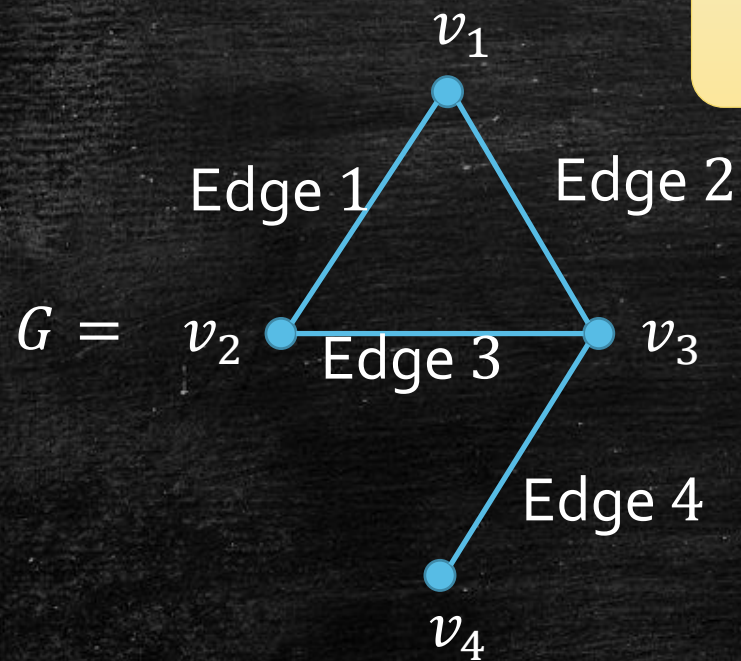
$$\mathbf{a}_3 = (1, 0, 1, 1, 1)$$

$$\mathbf{a}_4 = (1, 0, 0, 0, 1)$$

$$\mathbf{k} = (3, 1, 1, 1, 1)$$

a Vector Subset Sum instance

Example



Choosing two vertices becomes **infeasible!**

$$k = 3$$

a Vertex Cover instance

$$\mathbf{a}_1 = (1, 1, 1, 0, 0)$$

$$\mathbf{a}_2 = (1, 1, 0, 1, 0)$$

$$\mathbf{a}_3 = (1, 0, 1, 1, 1)$$

$$\mathbf{a}_4 = (1, 0, 0, 0, 1)$$

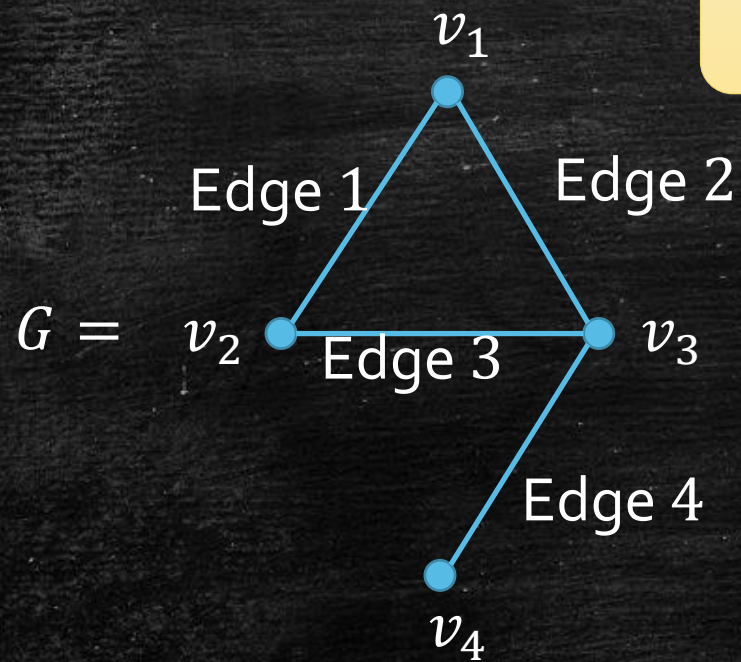
$$\mathbf{k} = (3, 1, 1, 1, 1)$$

a Vector Subset Sum instance

VertexCover $\leq_k$ VectorSubsetSum

- Task
 - Given a Vertex Cover instance $(G = (V, E), k)$.
 - Construct a Vector Subset Sum instance $(S, \mathbf{k})$.
- For each $v_i \in V$, construct a $(|E| + 1)$ -dimensional vector $\mathbf{a}_i \in S$ such that $\mathbf{a}_i[0] = 1$ and for each $j = 1, \dots, |E|$:
$$\mathbf{a}_i[j] = \begin{cases} 1 & \text{if } v_i \text{ is an endpoint of edge } j \\ 0 & \text{otherwise} \end{cases}$$
- For each edge j , construct $\mathbf{b}_j \in S$ where $\mathbf{b}_j[j] = 1$ is the only non-zero entry.
- Let $\mathbf{k} = (k, 2, 2, \dots, 2)$.

Example



$$k = 3$$

a Vertex Cover instance

We should choose
three vertices

$$\mathbf{a}_1 = (1, 1, 1, 0, 0)$$

$$\mathbf{a}_2 = (1, 1, 0, 1, 0)$$

$$\mathbf{a}_3 = (1, 0, 1, 1, 1)$$

$$\mathbf{a}_4 = (1, 0, 0, 0, 1)$$

$$\mathbf{b}_1 = (0, 1, 0, 0, 0)$$

$$\mathbf{b}_2 = (0, 0, 1, 0, 0)$$

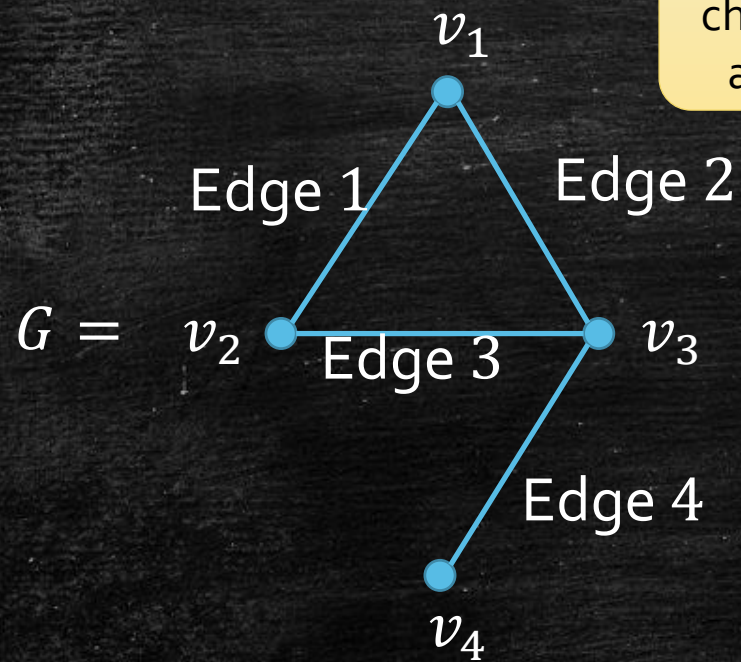
$$\mathbf{b}_3 = (0, 0, 0, 1, 0)$$

$$\mathbf{b}_4 = (0, 0, 0, 0, 1)$$

$$\mathbf{k} = (3, 2, 2, 2, 2)$$

a Vector Subset Sum instance

Example



$$k = 3$$

a Vertex Cover instance

We should either
choose two vertices or
a vertex + the edge.

$$\mathbf{a}_1 = (1, 1, 1, 0, 0)$$

$$\mathbf{a}_2 = (1, 1, 0, 1, 0)$$

$$\mathbf{a}_3 = (1, 0, 1, 1, 1)$$

$$\mathbf{a}_4 = (1, 0, 0, 0, 1)$$

$$\mathbf{b}_1 = (0, 1, 0, 0, 0)$$

$$\mathbf{b}_2 = (0, 0, 1, 0, 0)$$

$$\mathbf{b}_3 = (0, 0, 0, 1, 0)$$

$$\mathbf{b}_4 = (0, 0, 0, 0, 1)$$

$$\mathbf{k} = (3, 2, 2, 2, 2)$$

a Vector Subset Sum instance

Ideas Behind the Reduction

- Picking $\mathbf{a}_i \in T$ represents picking v_i in the vertex cover.
- The 0-th entry of $\mathbf{k}$ is set to k , enforcing exactly k vertices must be picked.
- The j -th entry of $\mathbf{k}$ is set to 2 enforcing edge j must be covered:
 - Say, edge j is (v_{i_1}, v_{i_2})
 - If $\mathbf{a}_{i_1}, \mathbf{a}_{i_2} \in T$, we are fine, as the j -th entries already add up to 2.
 - If one of $\mathbf{a}_{i_1}, \mathbf{a}_{i_2}$ is chosen in T , we are also fine, as we can include $\mathbf{b}_j \in T$.
 - If $\mathbf{a}_{i_1}, \mathbf{a}_{i_2} \notin T$, we are not fine: the j -th entries add up to at most 1 even if we include $\mathbf{b}_j \in T$.
- We are done! $\text{VertexCover} \leq_k \text{VectorSubsetSum}$

VectorSubsetSum $\leq_k$ SubsetSum

- We can convert a vector $\mathbf{a} = (a[0], \dots, a[m])$ to a large number.
- For example, convert $\mathbf{a} = (1, 4, 5, 3)$ to number 1453
 - $1453 = a[0] \times 1000 + a[1] \times 100 + a[2] \times 10 + a[3] \times 1$
- We are using decimal representation in the above example...
- To avoid carry, use N-ary representation instead (for sufficiently large N)?
- Additions with vectors are now equivalent to additions with numbers, since we do not have carry issue.
- VectorSubsetSum $\leq_k$ SubsetSum

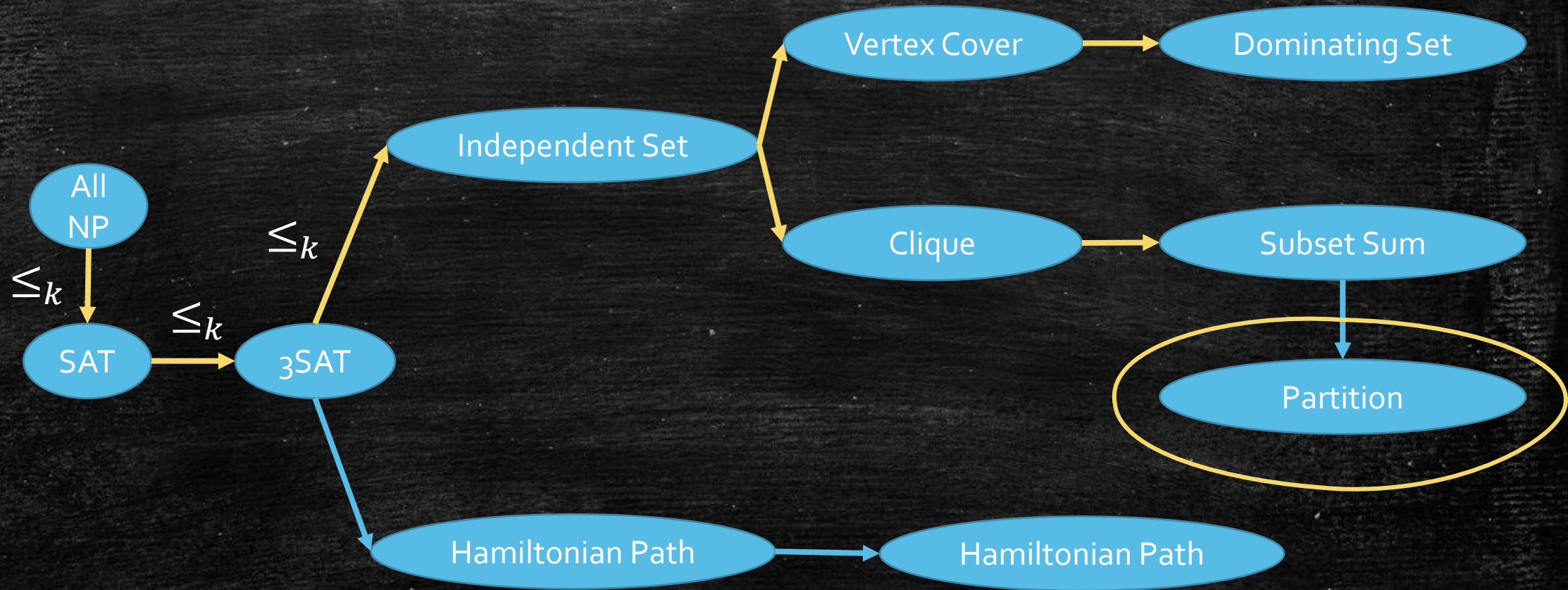
SubsetSum is NP-complete

- We have seen SubsetSum is in **NP**.
- We have proved
 1. $\text{VertexCover} \leq_k \text{VectorSubsetSum}$
 2. $\text{VectorSubsetSum} * \leq_k \text{SubsetSum}$

SubsetSum+

- **[SubsetSum+]** Given a collection of **positive** integers $S = \{a_1, \dots, a_n\}$ and $k \in \mathbb{Z}^+$, decide if there is a sub-collection $T \subseteq S$ such that $\sum_{a_i \in T} a_i = k$.
- SubsetSum+ is NP-complete
 - The same proof for SubsetSum can prove this!
- Test your "sense of direction": Which one holds trivially?
 - A. $\text{SubsetSum} \leq_k \text{SubsetSum+}$
 - B. $\text{SubsetSum+} \leq_k \text{SubsetSum}$

Our Reduction Graph



Partition Problem

- **[Partition]** Given a collection of integers S , decide if there is a partition of S to A and B such that $\sum_{a \in A} a = \sum_{b \in B} b$.
- **[Partition+]** Given a collection of **positive** integers S , decide if there is a partition of S to A and B such that $\sum_{a \in A} a = \sum_{b \in B} b$.
- Exercise: Prove that both Partition and Partition+ are NP-complete.

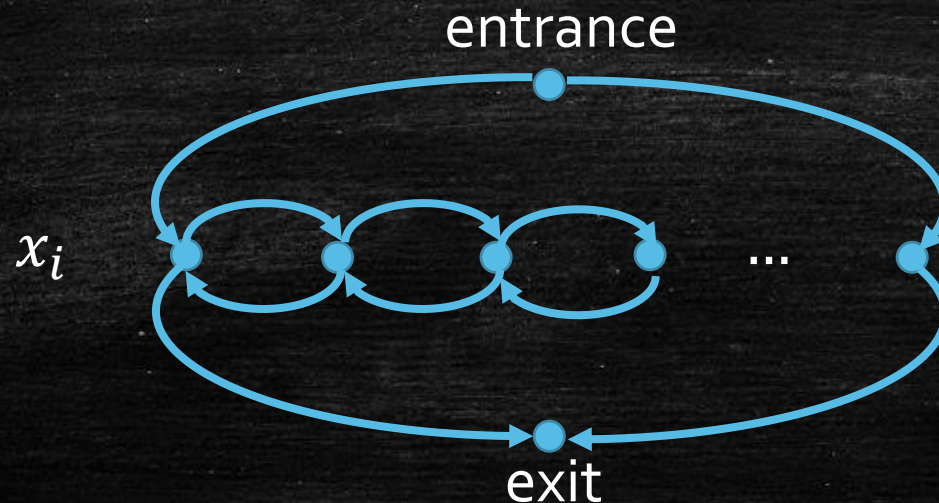
HamiltonianPath is NP-complete

- We have seen HamiltonianPath $\in$ **NP**. It remains to show its NP-hardness.
- Intermediate problem: DirectedHamiltonianPath
 - [DirectedHamiltonianPath] Given a **directed** graph $G = (V, E)$, a **source** $s \in V$ and a **sink** $t \in V$, decide if there is a Hamiltonian path from s to t .
- We will show:
 1. $3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$
 2. $\text{DirectedHamiltonianPath} \leq_k \text{HamiltonianPath}$

Note 7: constructing “gadgets” – be creative!

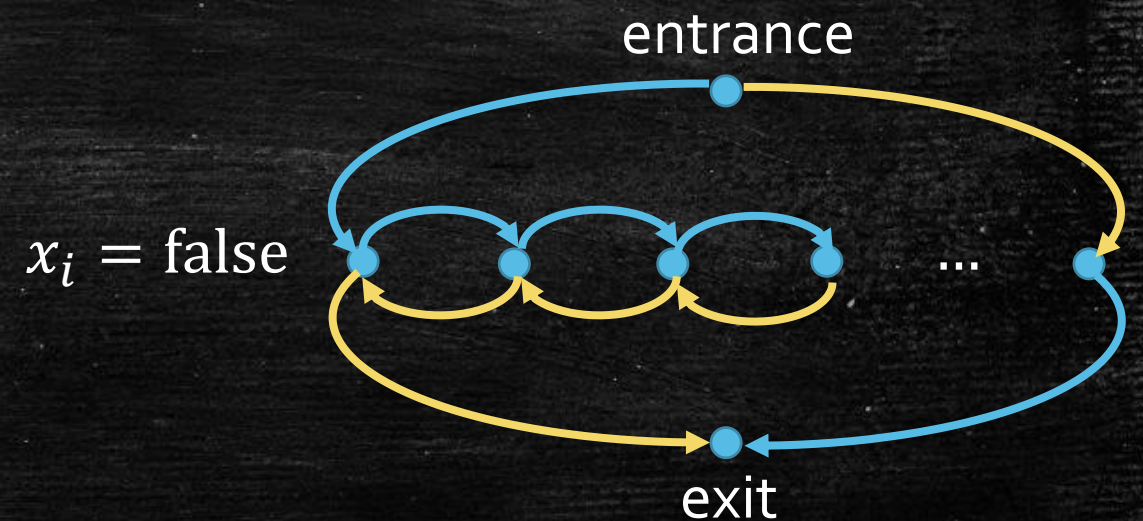
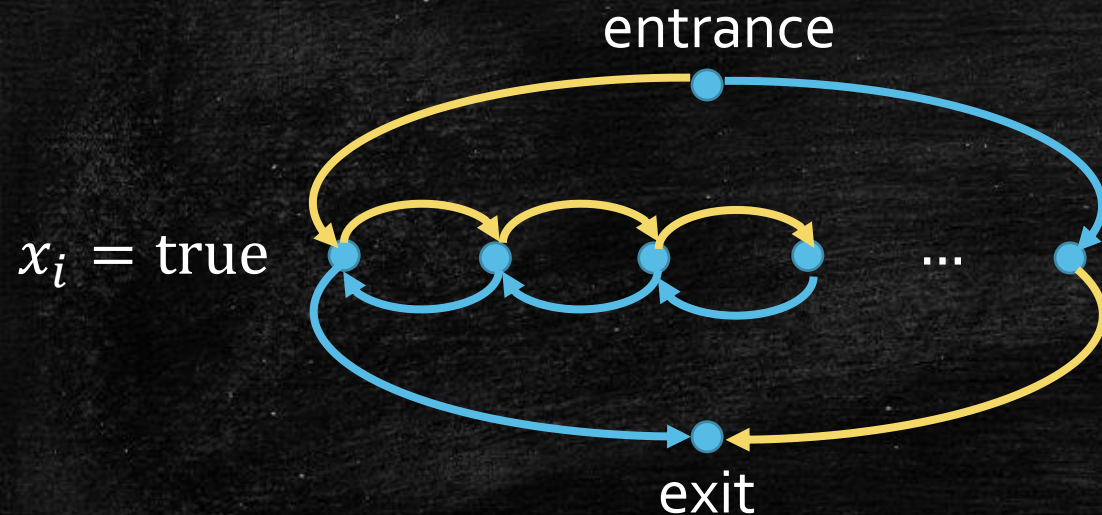
$3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$

- Given a 3SAT instance ϕ , we will construct a DirectedHamiltonianPath instance.
- Let n and m be the number of variables and clauses respectively.
- “Variable Gadget”



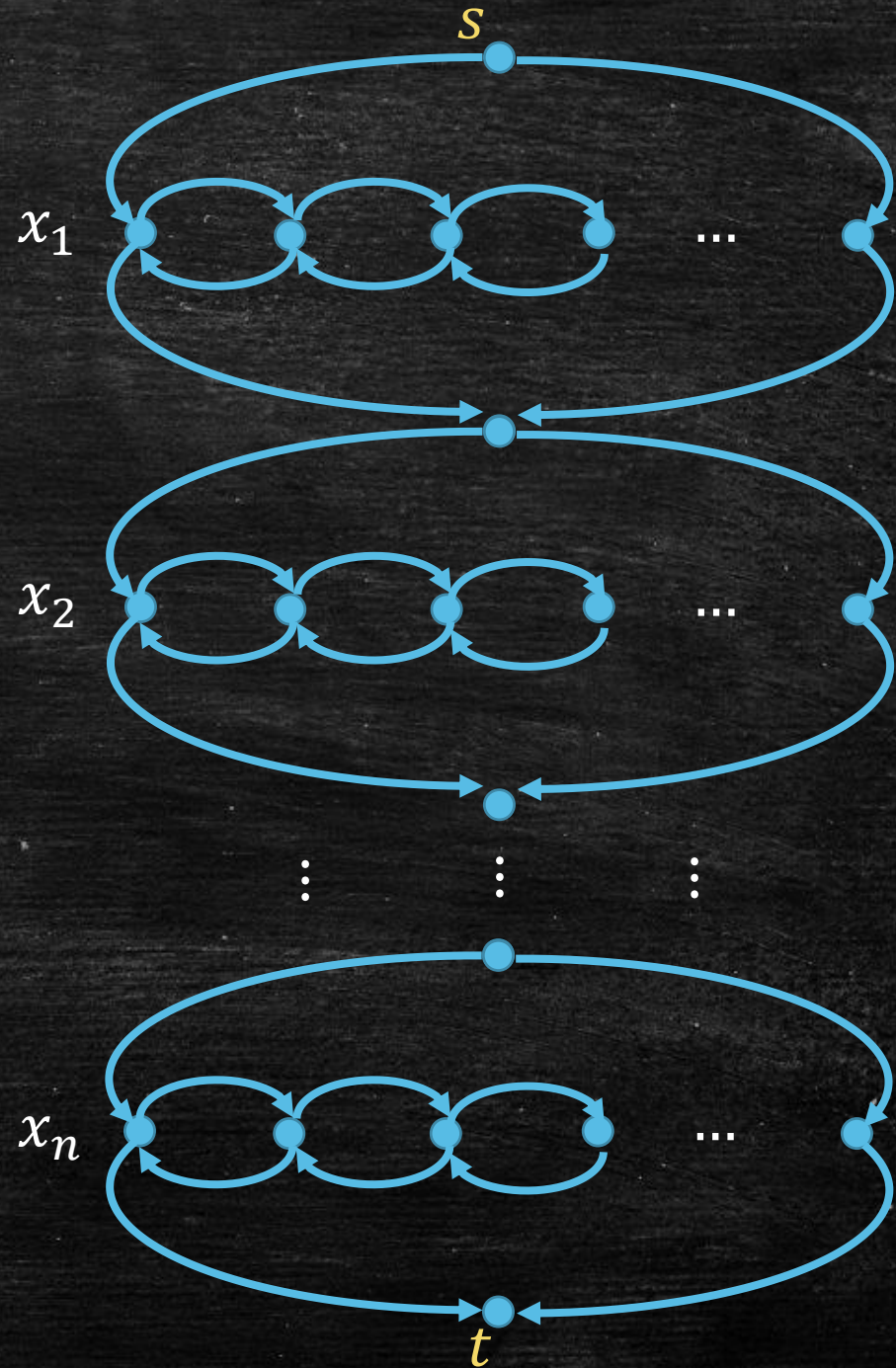
$3SAT \leq_k \text{DirectedHamiltonianPath}$

- There are two ways to go from "entrance" to "exit" that visit the middle vertices.
- They will represent $x_i = \text{true}$ and $x_i = \text{false}$ respectively.



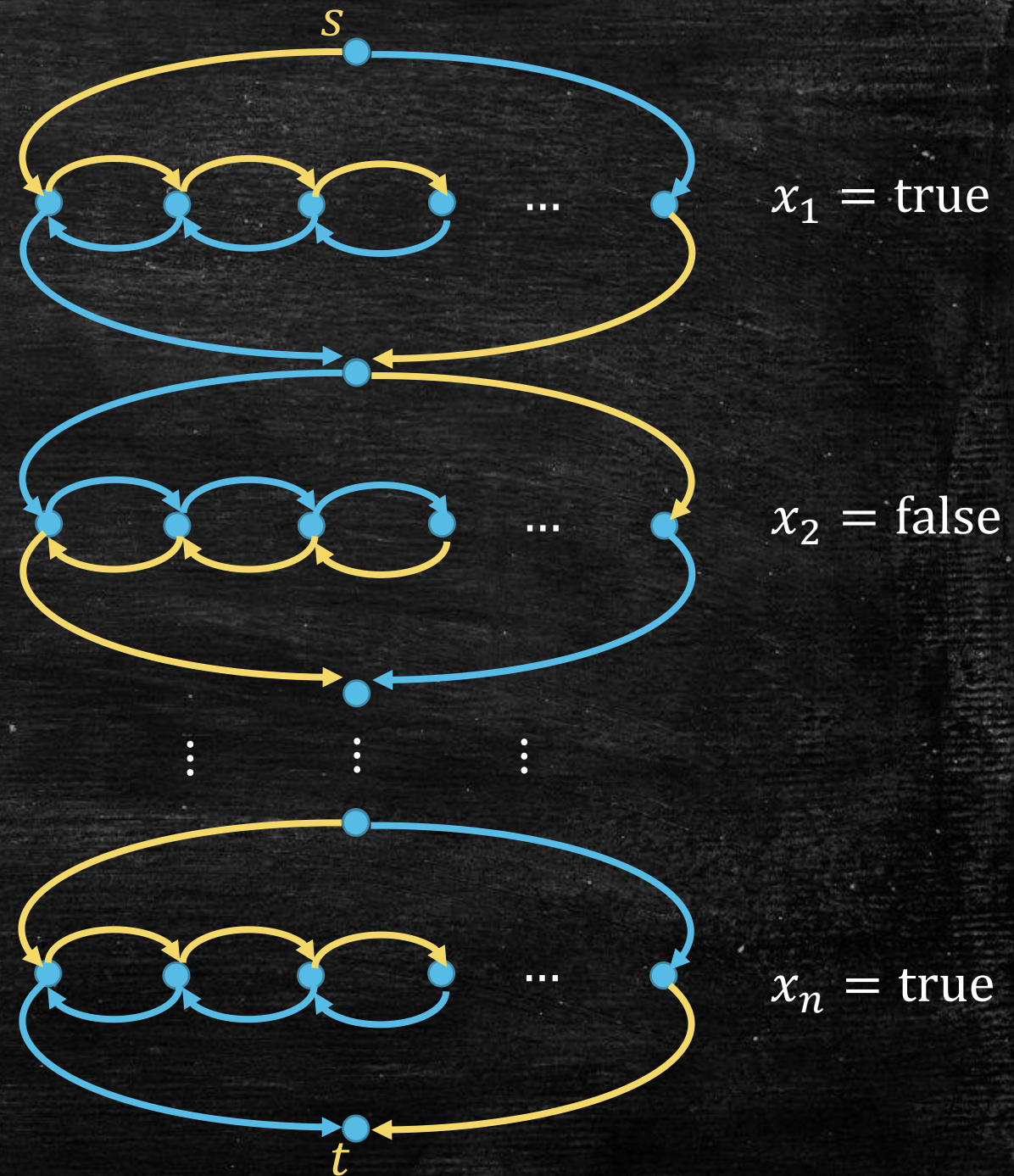
$3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$

- Connect all the variable gadgets.



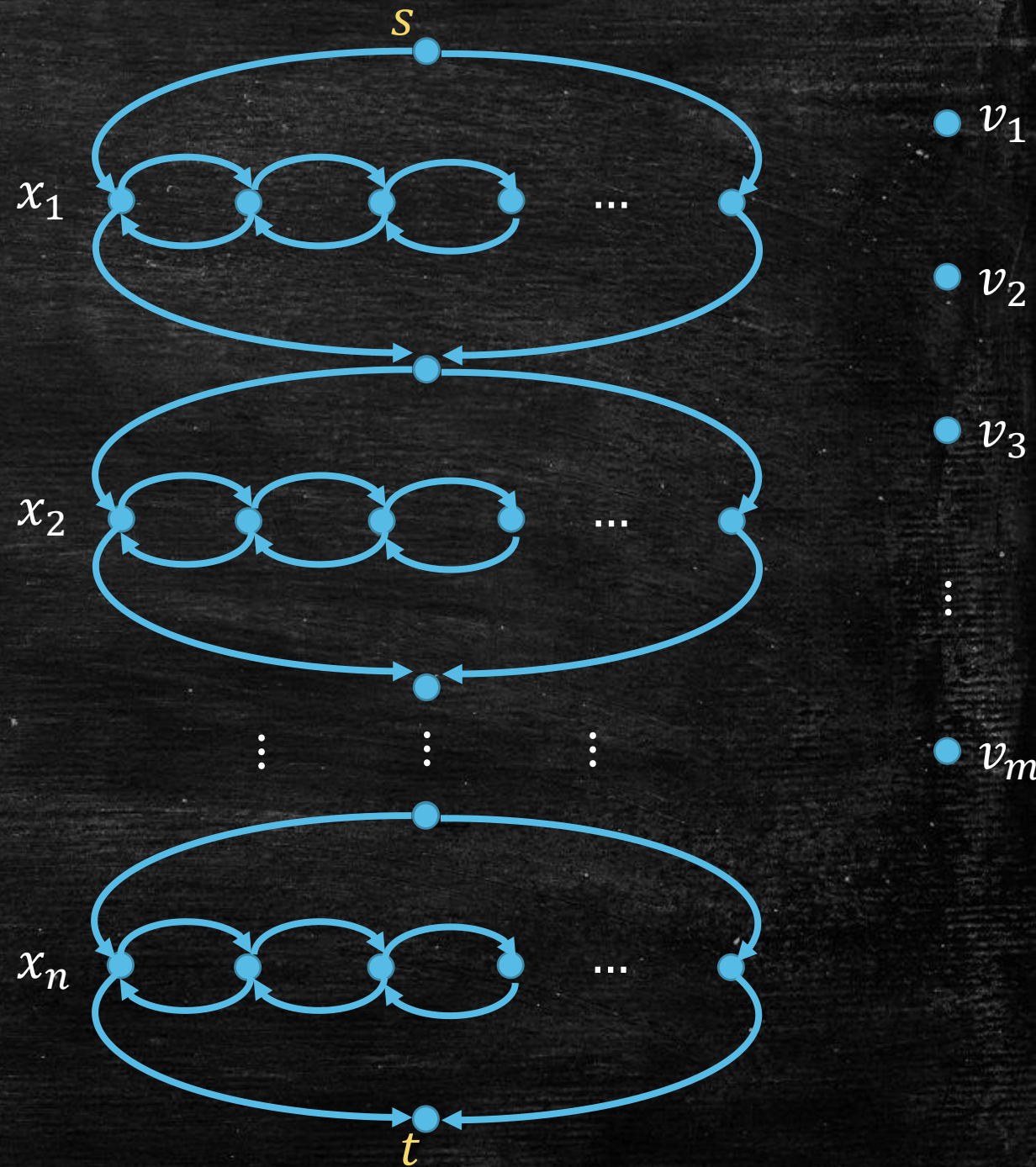
$3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$

- Connect all the variable gadgets.
- An s - t simple path visiting all middle vertices corresponds to an assignment to all variables.



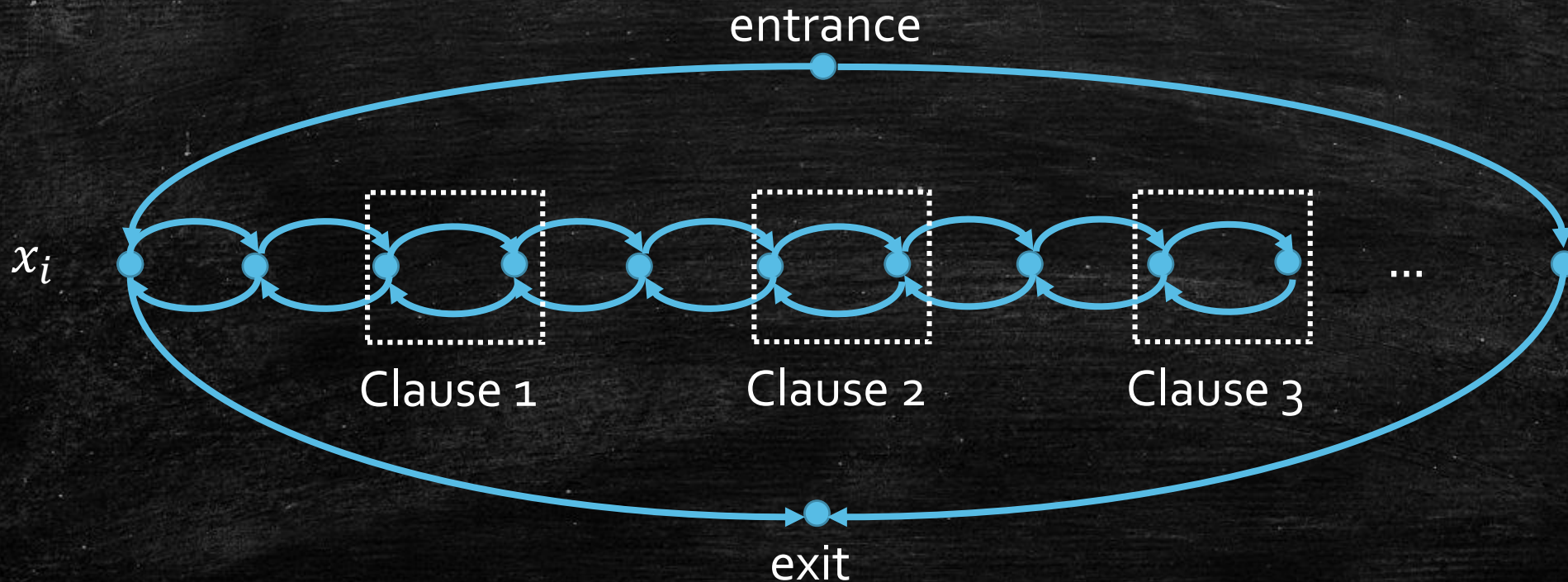
$3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$

- Connect all the variable gadgets.
- An s - t simple path visiting all middle vertices corresponds to an assignment to all variables.
- Build a vertex v_j for each clause j .



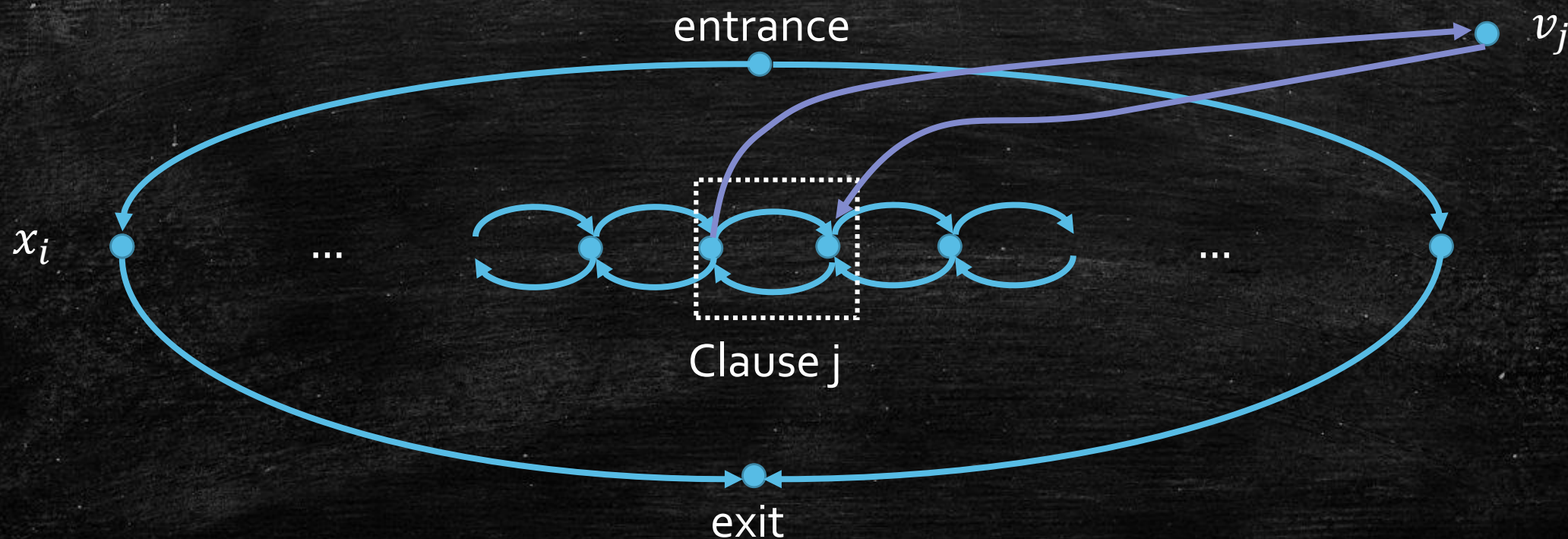
$3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$

- Inside the variable gadget, build $3m + 1$ middle vertices such that every two vertices corresponds to a clause separated by a "separator".



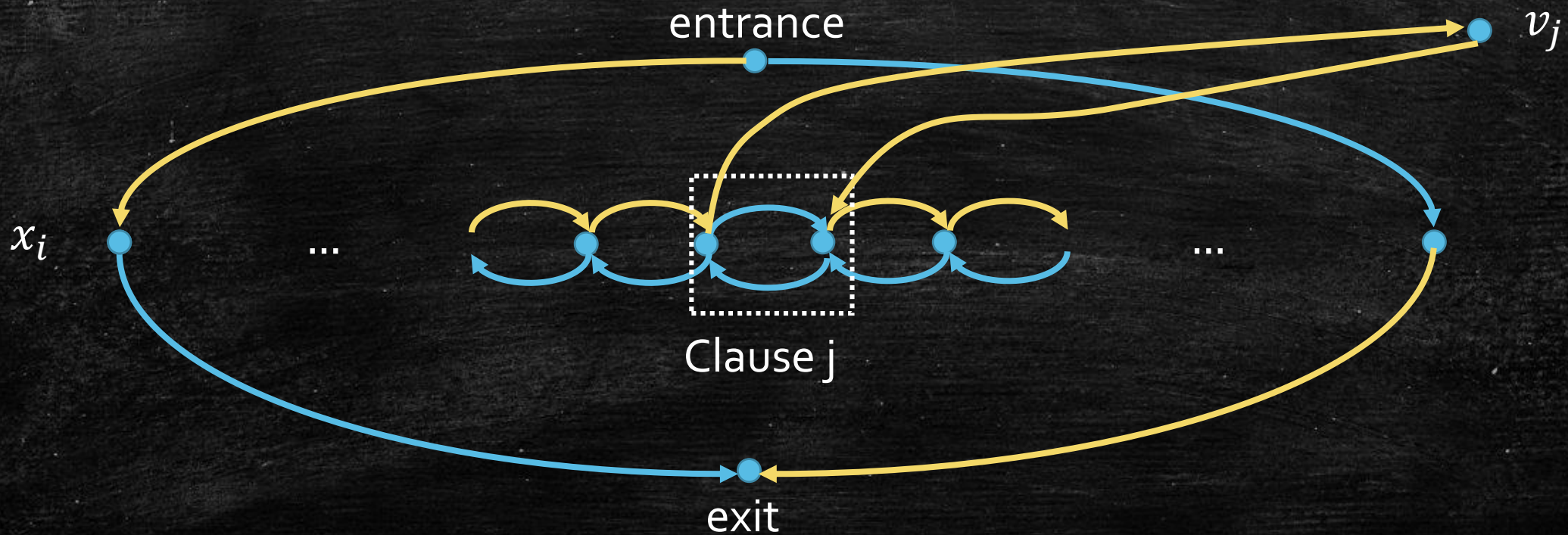
$3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$

- If x_i is in j -th clause, connect the gadget to v_j as follows.



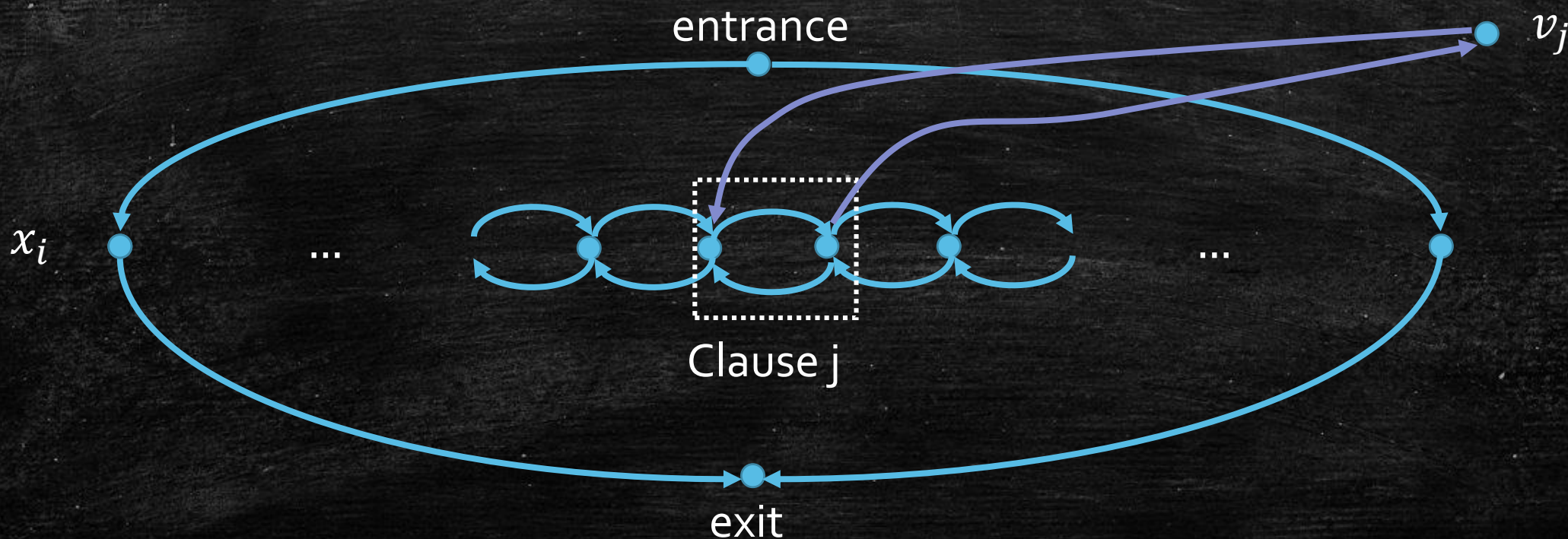
$3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$

- If x_i is in j -th clause, connect the gadget to v_j as follows.
- If $x_i = \text{true}$, j -th clause is satisfied, we can take a detour and visit v_j .



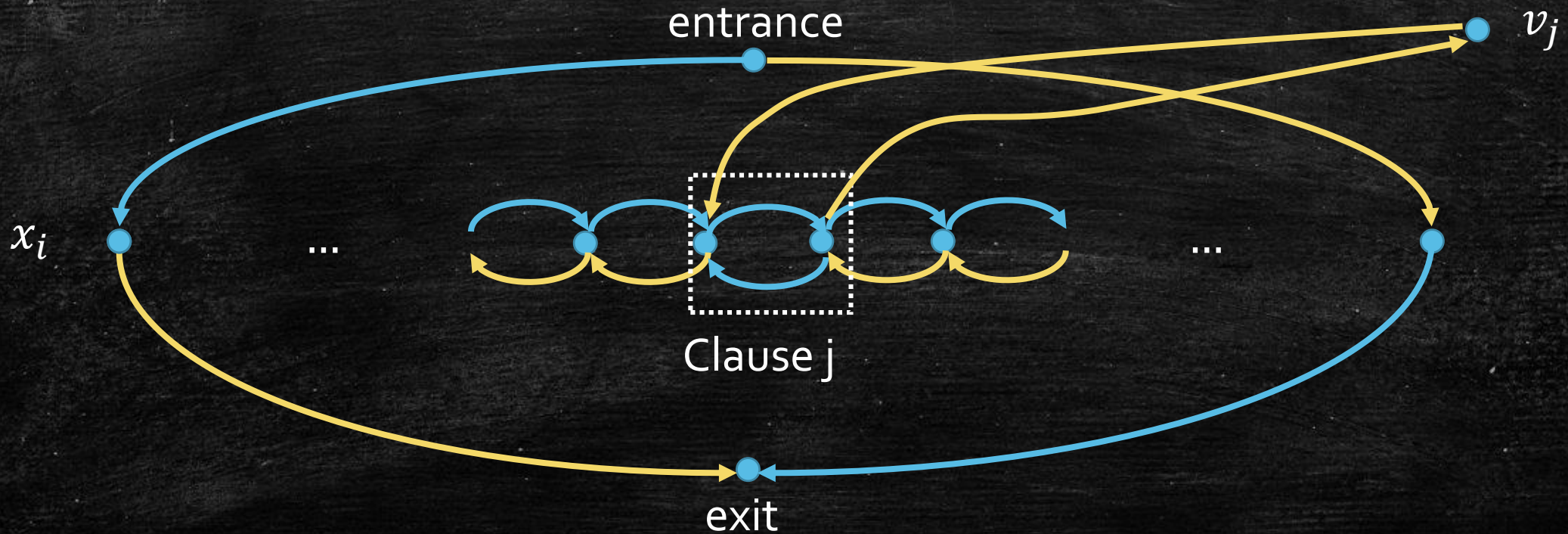
$3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$

- If $\neg x_i$ is in j -th clause, connect the gadget to v_j as follows.



$3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$

- If $\neg x_i$ is in j -th clause, connect the gadget to v_j as follows.
- If $x_i = \text{false}$, j -th clause is satisfied, we can take a detour and visit v_j .



$3\text{SAT} \leq_k \text{DirectedHamiltonianPath}$

If ϕ is a yes instance, the graph has a Hamiltonian path:

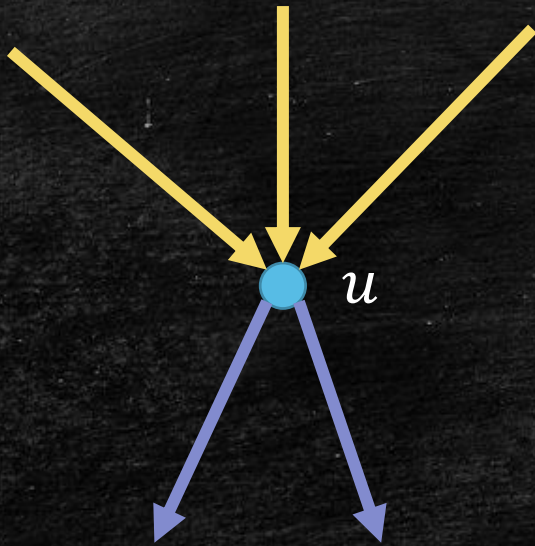
- For each clause, choose a representative true literature.
- Go from s to t , and visit each v_j from its representative by taking a detour.

If the graph has a Hamiltonian path, ϕ is a yes instance:

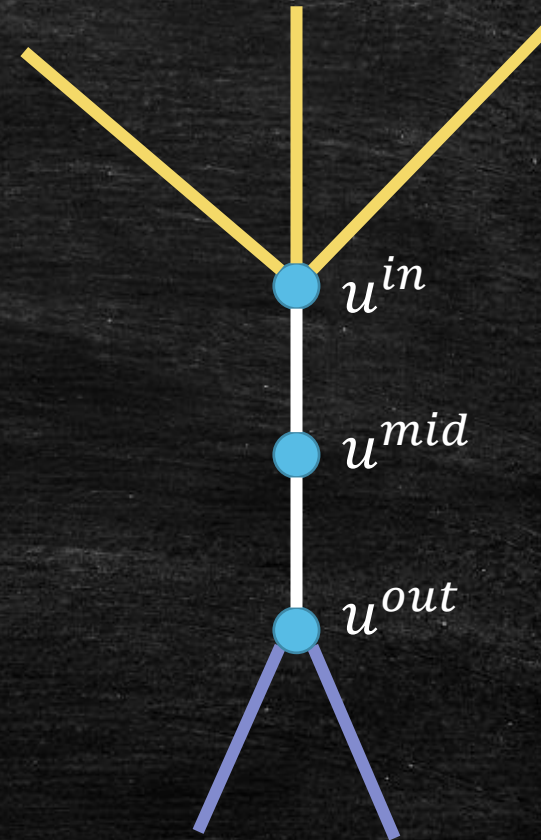
- The Hamiltonian path has to go from s to t .
- Each v_j has to be visited by a detour from a variable.
- The variable's value is then determined.

DirectedHamiltonianPath $\leq_k$ HamiltonianPath

- Vertex Gadget:



a vertex and its incident edges
DirectedHamiltonianPath instance



a **vertex gadget** and its incident edges
HamiltonianPath instance

DirectedHamiltonianPath $\leq_k$ HamiltonianPath

If G is a yes DirectedHamiltonianPath instance, G' is a yes HamiltonianPath instance:

- Hamiltonian path in G : $s \rightarrow u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_n \rightarrow t$
- Hamiltonian path in G' : $s^{in} \rightarrow s^{mid} \rightarrow s^{out} \rightarrow u_1^{in} \rightarrow u_1^{mid} \rightarrow u_1^{out} \rightarrow u_2^{in} \rightarrow \dots \rightarrow u_n^{out} \rightarrow t^{in} \rightarrow t^{mid} \rightarrow t^{out}$

DirectedHamiltonianPath $\leq_k$ HamiltonianPath

If G' is a yes HamiltonianPath instance, G is a yes DirectedHamiltonianPath instance:

Show that the yes HamiltonianPath instance is "well-behaved"

- **Lemma 1.** The path in G' must start at s^{in} and end at t^{out} .

DirectedHamiltonianPath $\leq_k$ HamiltonianPath

If G' is a yes HamiltonianPath instance, G is a yes DirectedHamiltonianPath instance:

Show that the yes HamiltonianPath instance is "well-behaved"

- **Lemma 1.** The path in G' must start at s^{in} and end at t^{out} .
- Proof. s^{in} and t^{out} have degree 1, so they must be starting and ending vertices.
- We can assume the path goes from s^{in} to t^{out}
 - Going from t^{out} to s^{in} is equivalent, as the graph is undirected.

DirectedHamiltonianPath $\leq_k$ HamiltonianPath

If G' is a yes HamiltonianPath instance, G is a yes DirectedHamiltonianPath instance:

Show that the yes HamiltonianPath instance is "well-behaved"

- **Lemma 1.** The path in G' must start at s^{in} and end at t^{out} .
- **Lemma 2.** If we first enter a vertex gadget at u^{in} (or u^{out}) we must proceed to u^{mid} and then to u^{out} (or u^{in}).

DirectedHamiltonianPath $\leq_k$ HamiltonianPath

If G' is a yes HamiltonianPath instance, G is a yes DirectedHamiltonianPath instance:

Show that the yes HamiltonianPath instance is "well-behaved"

- **Lemma 1.** The path in G' must start at s^{in} and end at t^{out} .
- **Lemma 2.** If we first enter a vertex gadget at u^{in} (or u^{out}) we must proceed to u^{mid} and then to u^{out} (or u^{in}).
- **Proof.** If we go to u^{in} and do not proceed to u^{mid} , we have nowhere to go when we reach u^{mid} in the future.
- u^{mid} must be an endpoint of the path, contradicting to Lemma 1.

DirectedHamiltonianPath $\leq_k$ HamiltonianPath

If G' is a yes HamiltonianPath instance, G is a yes DirectedHamiltonianPath instance:

Show that the yes HamiltonianPath instance is "well-behaved"

- **Lemma 1.** The path in G' must start at s^{in} and end at t^{out} .
- **Lemma 2.** If we first enter a vertex gadget at u^{in} (or u^{out}) we must proceed to u^{mid} and then to u^{out} (or u^{in}).
- **Lemma 3.** The pattern of the path must be $in \rightarrow mid \rightarrow out \rightarrow in \rightarrow mid \rightarrow out \rightarrow \dots$

DirectedHamiltonianPath $\leq_k$ HamiltonianPath

If G' is a yes HamiltonianPath instance, G is a yes DirectedHamiltonianPath instance:

Show that the yes HamiltonianPath instance is "well-behaved"

- **Lemma 1.** The path in G' must start at s^{in} and end at t^{out} .
- **Lemma 2.** If we first enter a vertex gadget at u^{in} (or u^{out}) we must proceed to u^{mid} and then to u^{out} (or u^{in}).
- **Lemma 3.** The pattern of the path must be $in \rightarrow mid \rightarrow out \rightarrow in \rightarrow mid \rightarrow out \rightarrow \dots$
 - Proof. We start at s^{in} (Lemma 1) and we must go to s^{mid} and s^{out} (Lemma 2).
 - Each u^{out} is only connected to an v^{in} , and we need to proceed to v^{mid} and v^{out} (Lemma 2).

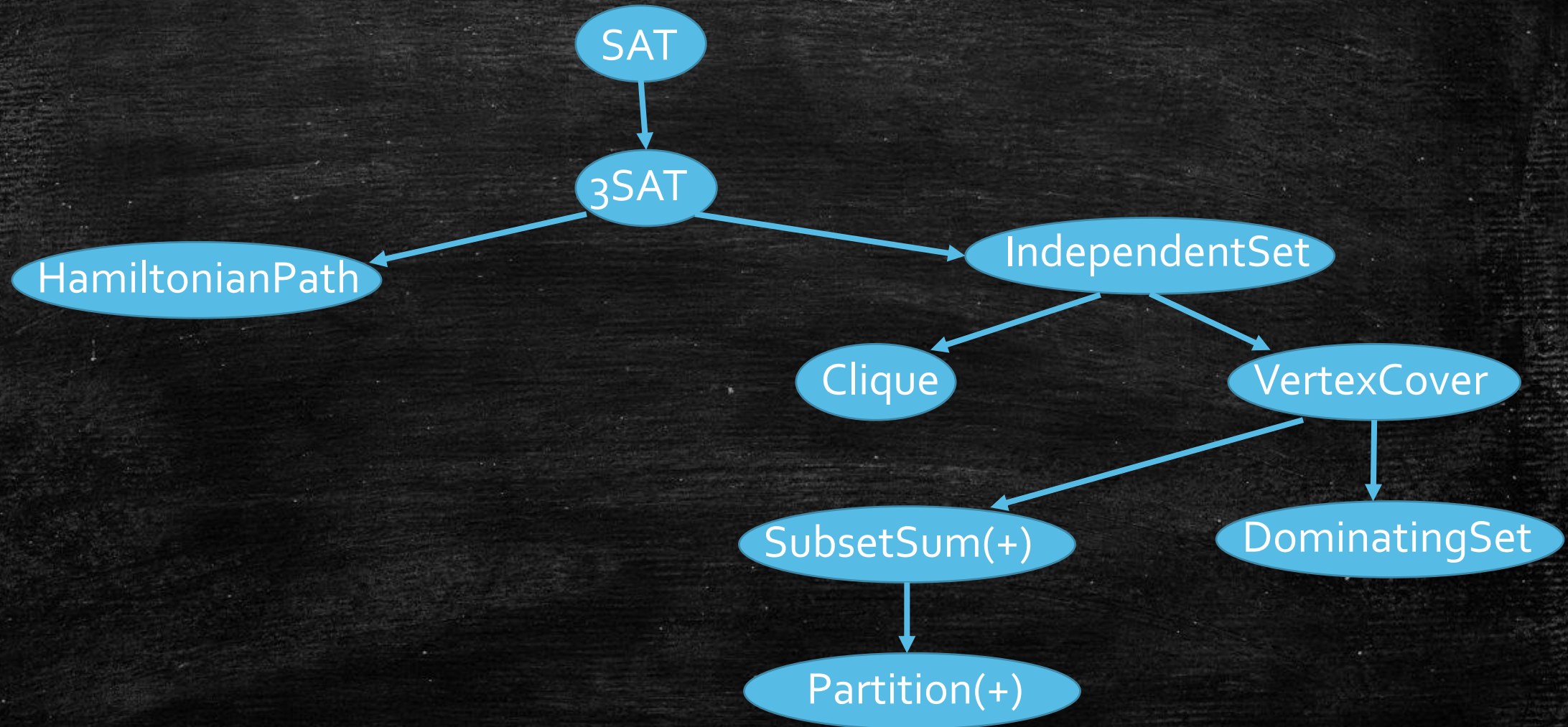
DirectedHamiltonianPath $\leq_k$ HamiltonianPath

If G' is a yes HamiltonianPath instance, G is a yes DirectedHamiltonianPath instance:

Show that the yes HamiltonianPath instance is "well-behaved"

- **Lemma 1.** The path in G' must start at s^{in} and end at t^{out} .
- **Lemma 2.** If we first enter a vertex gadget at u^{in} (or u^{out}) we must proceed to u^{mid} and then to u^{out} (or u^{in}).
- **Lemma 3.** The pattern of the path must be $in \rightarrow mid \rightarrow out \rightarrow in \rightarrow mid \rightarrow out \rightarrow \dots$
- Now we have a Hamiltonian path in G' corresponding to a path in G .

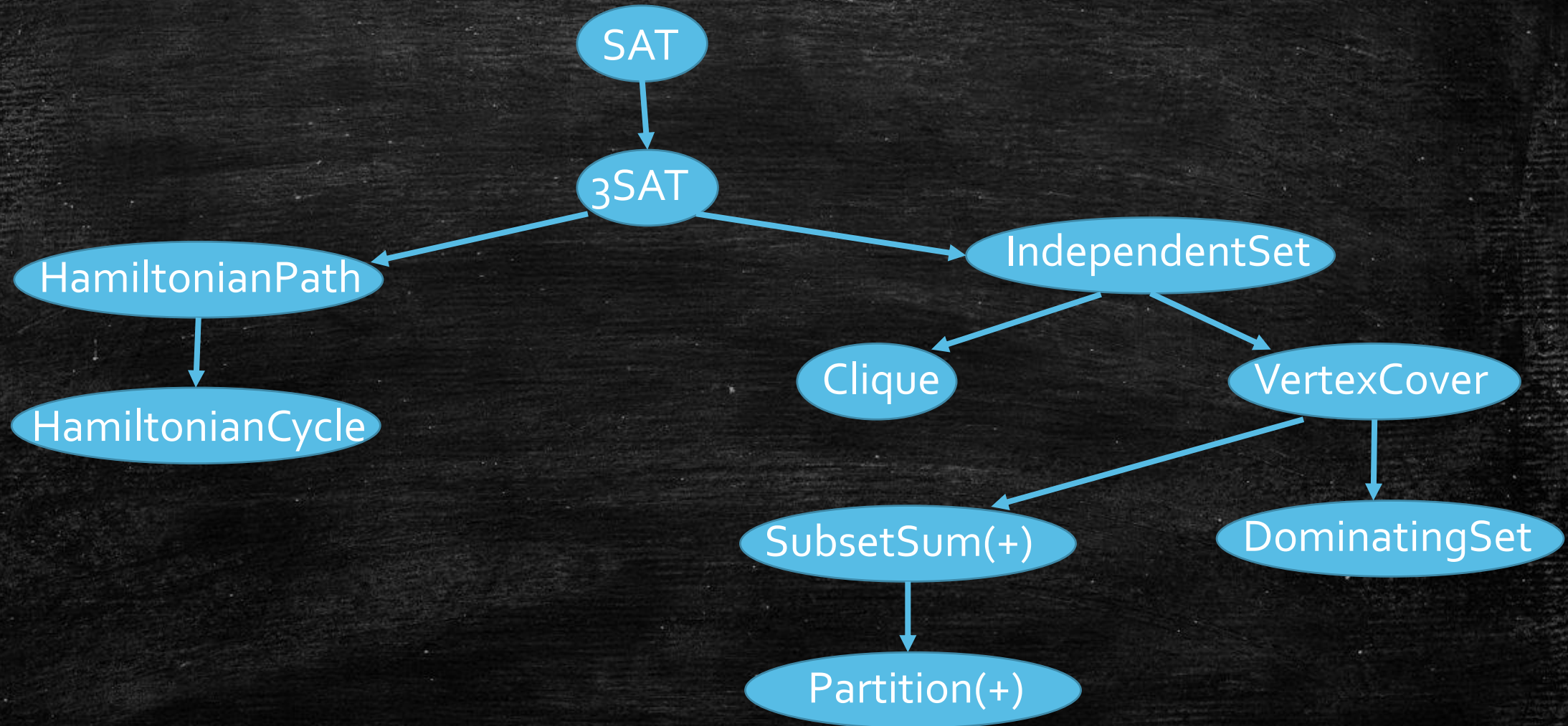
Web of NP-complete Problems



Hamiltonian Cycle

- Given an undirected graph $G = (V, E)$, a **Hamiltonian cycle** is a cycle that visits each vertex exactly once.
- [HamiltonianCycle] Given an undirected graph $G = (V, E)$, decide if G contains a Hamiltonian cycle.
- Exercise: Prove that HamiltonianCycle is NP-complete.

Web of NP-complete Problems



Five Most Important NP-Complete Problems

Most NP-complete problems can be reduced from...

- 3SAT
- IndependentSet (Clique)
- VertexCover
- SubsetSum (Partition)
- HamiltonianPath (HamiltonianCycle)

Techniques we have seen...

1. Choose the right problem to reduce from
2. Fix the reduction by minor modifications
3. Show the contrapositive for the mapping of no instances
4. Show the yes instance being reduced to is "well-behaved"
5. Do not mess-up the direction
6. Introduce intermediate problems
7. Use gadgets – be creative

NP-Hard vs NP-Complete

Difference between NP-hardness and NP-completeness:

- For decision problems: NP-complete = NP-hard + (in **NP**)
 - There are NP-hard problems that are not in **NP**; these problems are even harder than NP-complete problems.
- NP-hardness can describe optimization/search problems

NP-hard Optimization Problems (Informal)

- A **maximization** problem is NP-hard if there exists $k \in \mathbb{R}$ such that **deciding** whether $\text{OPT} \geq k$ is NP-hard.
- A **minimization** problem is NP-hard if there exists $k \in \mathbb{R}$ such that **deciding** whether $\text{OPT} \leq k$ is NP-hard.
- If there exists a polynomial time algorithm to solve an NP-hard optimization problem, then **P = NP**.
 - If OPT can be computed in polynomial time, whether $\text{OPT} \geq k$ ($\text{OPT} \leq k$) can also be decided in polynomial time.
 - Solving an NP-hard decision problem in polynomial time implies **P = NP**.

NP-hard Optimization Problem Examples

- **[Max-3SAT]** Maximizing the number of satisfying clauses.
 - NP-hard to decide if $\text{OPT} \geq \text{NumOfClauses}$
- **[Max-IndependentSet]** Maximizing the size of the independent set.
 - NP-hard to decide if $\text{OPT} \geq k$
 - Note: existence of k -independent set implies $\text{OPT} \geq k$.
- **[Min-VertexCover]** Minimizing the size of the vertex cover.
 - NP-hard to decide if $\text{OPT} \leq k$
 - Note: existence of k -vertex cover implies $\text{OPT} \leq k$.
- **[LongestPath]** Maximizing the length of a simple path.
 - NP-hard to decide if $\text{OPT} \geq |V|$ (HamiltonianPath)

Makespan Minimization (Revisited)

- Makespan Minimization is NP-hard.
- Let $k = \frac{1}{2} \sum_{i=1}^n p_i$.
- For even two machines, it is NP-hard to decide whether optimal makespan $\leq k$.
- An obvious reduction from Partition.

Travelling Salesman Problem (TSP)

- [TSP] Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?
- [TSP (Formulation)] Given a weighted and complete undirected graph $G = (V, E = V \times V, w)$, find a Hamiltonian cycle with minimum length.
- Differences with HamiltonianCycle:
 - A Hamiltonian cycle always exists for TSP
 - But the graph is weighted, we need to optimize the path length

TSP is NP-hard

- Given a HamiltonianPath instance $G = (V, E)$, we construct a TSP instance $G = (V', E', w)$ such that
 - $V' = V$
 - $w(u, v) = 1$ if $(u, v) \in E$
 - $w(u, v) = |V|^{2615}$ is a very large number if $(u, v) \notin E$
- It's NP-hard to decide if optimal tour has length at most $|V|$.

TSP is even hard to “approximate”!

- **Theorem.** Suppose $P \neq NP$. There is no polynomial time α -approximation algorithm for TSP for any $\alpha \geq 1$ that may depend on the instance.
- Theorem holds for exponentially large α , e.g., $\alpha = (2615|V|)^{2615|V|}$.
- Proof. Change $|V|^{2615}$ to $\alpha|V| + 1$ in the previous reduction.
- Yes HamiltonianCycle instance $\Rightarrow \text{OPT}_{\text{TSP}} = |V|$
- No HamiltonianCycle instance $\Rightarrow \text{OPT}_{\text{TSP}} \geq \alpha|V| + 1$
- Let ALG be the output of an α -approximation algorithm $\mathcal{A}$.
- $\text{ALG} \leq \alpha|V| \Rightarrow$ yes HamiltonianCycle instance
- $\text{ALG} \geq \alpha|V| + 1 \Rightarrow$ no HamiltonianCycle instance

This Lecture

- Show more important NP-complete problems.
 - DominatingSet
 - SubsetSum (Partition)
 - HamiltonianPath (HamiltonianCycle)
- Learn some elementary techniques for reduction.
- Learn how to write a formal proof for NP-completeness.
- NP-hard optimization problems
 - Makespan Minimization
 - TSP